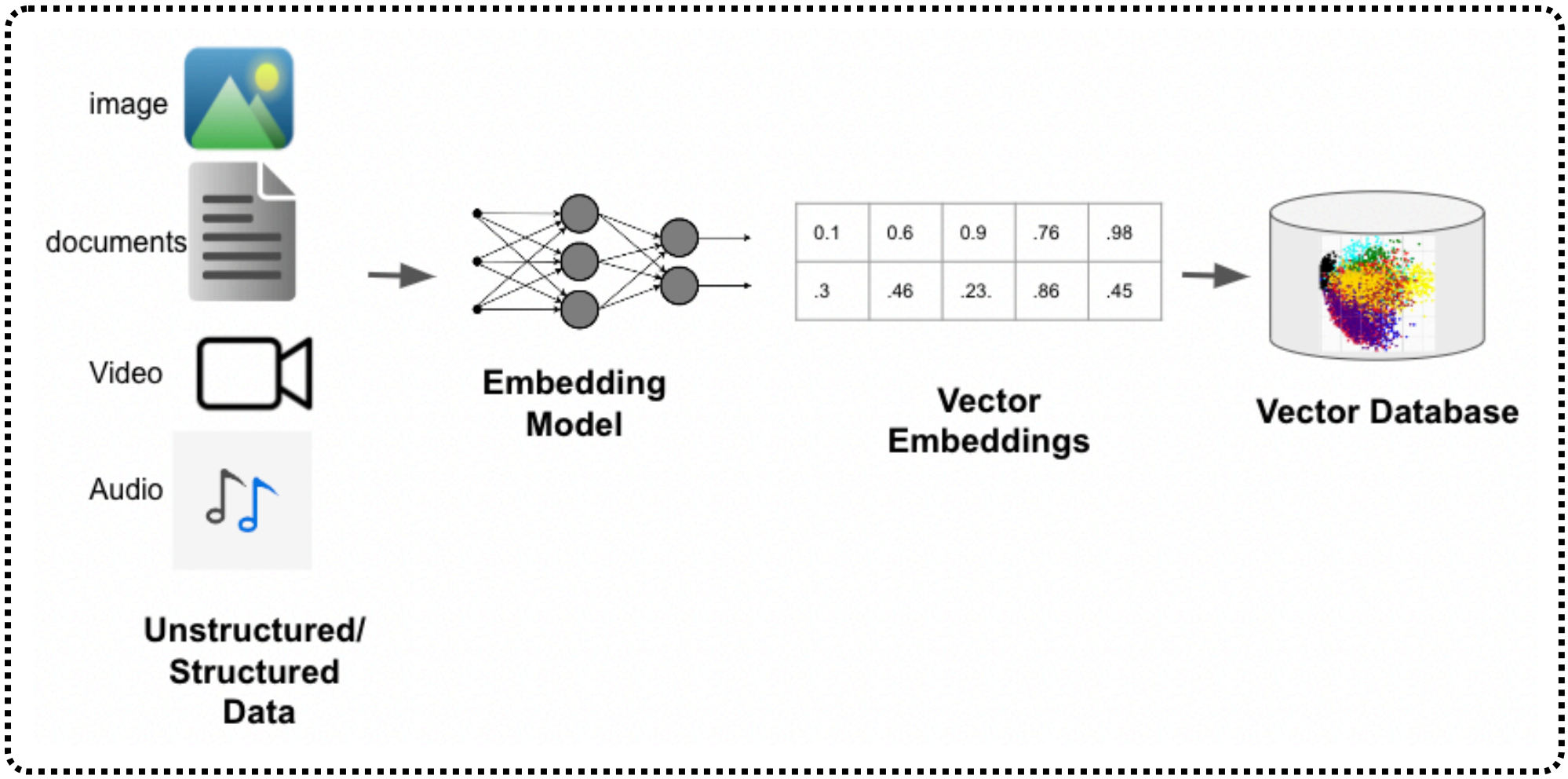
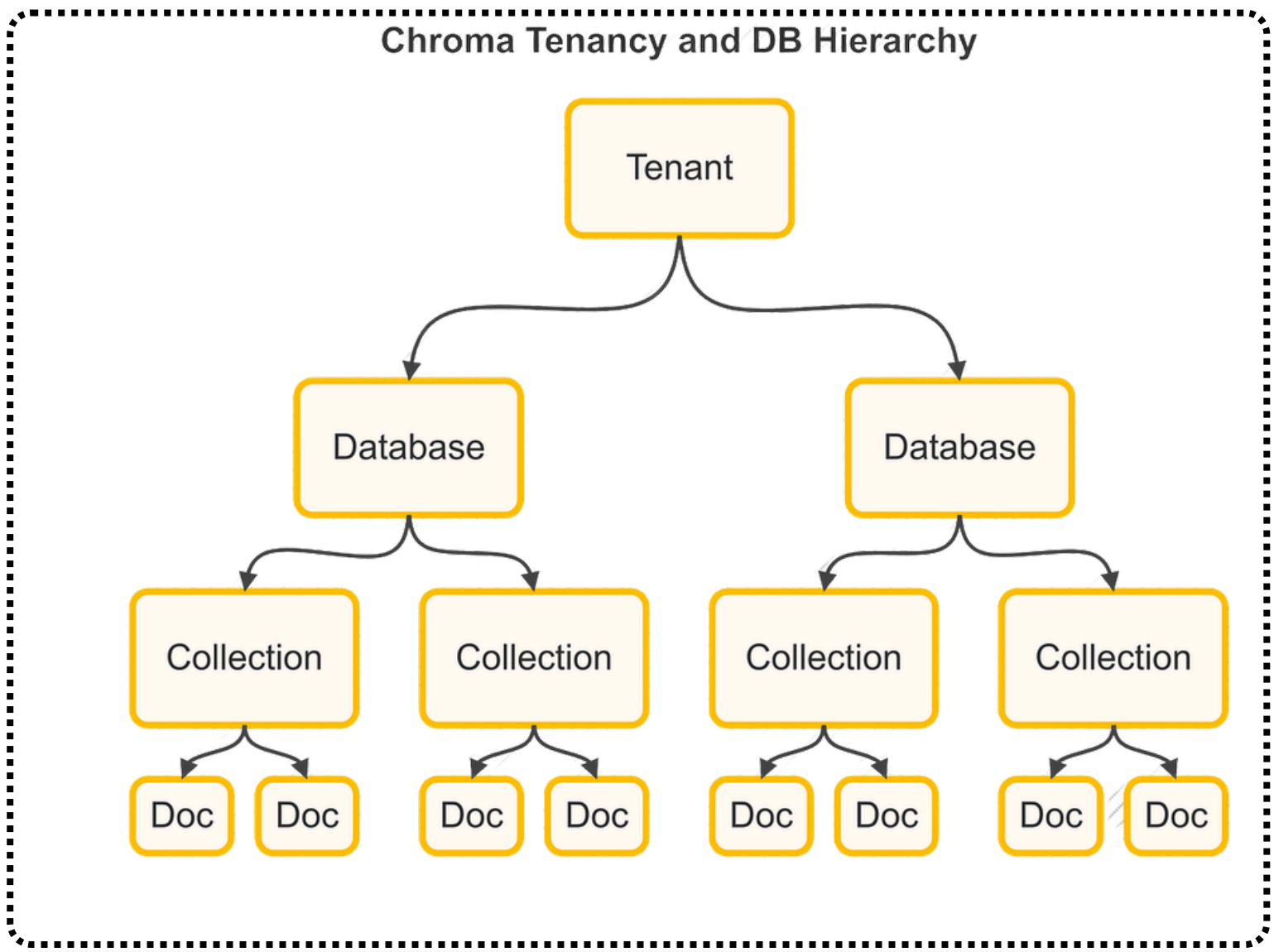
# Complete Notes on Chroma

## Free Course



A **compact, saveable reference** covering the core concepts, architecture, and usage of **ChromaDB**, widely used for building **vector search, RAG pipelines, and AI memory systems**.

## 1. What is ChromaDB

**ChromaDB** is an **open-source vector database designed for AI applications**. It stores **embeddings (vector representations of data)** and allows fast **similarity search**.

It is primarily used for:

- Retrieval-Augmented Generation (RAG)
- Semantic search
- AI memory systems
- document retrieval
- recommendation systems
- AI agents

Core idea:

```
Plain text

Data → Embeddings → Vector Database → Similarity Search
```

---

## 2. Why ChromaDB is Used in AI Systems

Large language models cannot store knowledge internally for every application.

ChromaDB enables:

- storing embeddings
- retrieving relevant context
- grounding LLM responses

Typical pipeline:

```
Plain text

User Query
   ↓
Embedding Model
   ↓
Vector Search (ChromaDB)
   ↓
Relevant Documents
   ↓
LLM Response
```

## 3. Key Features

| Feature | Description |
|---|---|
| Vector storage | stores embeddings |
| Similarity search | finds nearest vectors |
| Metadata filtering | structured filtering |
| Persistence | local or server storage |
| Lightweight | easy to deploy |
| Python & JS support | flexible integration |

---

## 4. Core Components of ChromaDB

### 1. Client

Interface to interact with the database.

Example:

```python
import chromadb

client = chromadb.Client()
```

---

### 2. Collections

Collections are similar to **tables in databases**.

They store:

- embeddings
- documents
- metadata
- IDs

Example:

```python
collection = client.create_collection(name="documents")
```

---

### 3. Embeddings

Vectors representing text, images, or other data.

Example embedding size:

```
384
768
1536
```

### 4. Documents

The original content stored with embeddings.

Example:

```
"LangChain is a framework for building LLM applications."
```

---

### 5. Metadata

Additional structured information.

Example:

```python
{
 "source": "research_paper",
 "author": "John"
}
```

---

## 5. ChromaDB Architecture

Basic architecture:

```
Application
   ↓
Embedding Model
   ↓
ChromaDB
   ↓
Vector Index
   ↓
Similarity Search
```

Key layers:

| Layer | Purpose |
|---|---|
| Storage layer | stores vectors |
| Index layer | enables fast retrieval |
| Query engine | similarity search |

## 5. ChromaDB Architecture

Basic architecture:

```plain text
Application
    ↓
Embedding Model
    ↓
ChromaDB
    ↓
Vector Index
    ↓
Similarity Search
```

Key layers:

| Layer | Purpose |
|---|---|
| Storage layer | stores vectors |
| Index layer | enables fast retrieval |
| Query engine | similarity search |

## 6. Installing ChromaDB

Install using pip:

```python
pip install chromadb
```

Optional dependencies:

```python
pip install sentence-transformers
```

## 7. Creating a ChromaDB Client

Example:

```python
import chromadb

client = chromadb.Client()
```

Persistent client:

```python
from chromadb.config import Settings

client = chromadb.Client(
    Settings(persist_directory="./chroma")
)
```

## 8. Creating a Collection

Example:

```python
collection = client.create_collection(
    name="my_collection"
)
```

Get existing collection:

```python
collection = client.get_collection("my_collection")
```

## 9. Adding Data to ChromaDB

Example:

```python
collection.add(
    documents=["AI is transforming industries"],
    metadatas=[{"topic": "AI"}],
    ids=["doc1"]
)
```

You can also add embeddings manually.

Example:

```python
collection.add(
    embeddings=[[0.1, 0.2, 0.3]],
    ids=["vector1"]
)
```

## 10. Querying the Database

Example similarity search:

```python
results = collection.query(
    query_texts=["What is AI?"],
    n_results=2
)
```

Output:

```python
{
 "documents": [...],
 "ids": [...],
 "distances": [...]
}
```

## 11. Filtering with Metadata

You can filter search results using metadata.

Example:

```python
results = collection.query(
    query_texts=["AI applications"],
    where={"topic": "AI"}
)
```

Useful for:

- document filtering
- domain restriction
- category search

---

## 12. Deleting Data

Delete specific items:

```python
collection.delete(ids=["doc1"])
```

Delete entire collection:

```python
client.delete_collection("my_collection")
```

---

## 13. Updating Documents

Example:

```python
collection.update(
    ids=["doc1"],
    documents=["Updated AI content"]
)
```

---

## 14. Similarity Search

ChromaDB retrieves vectors using **distance metrics**.

Common metrics:

| Metric | Description |
|---|---|
| Cosine similarity | angle similarity |
| Euclidean distance | geometric distance |
| Dot product | vector multiplication |

Cosine similarity is most common in AI systems.

## 15. Embedding Models Used with ChromaDB

Common embedding models:

| Model | Provider |
|---|---|
| text-embedding-3-small | OpenAI |
| text-embedding-3-large | OpenAI |
| all-MiniLM-L6-v2 | Sentence Transformers |
| Instructor XL | HuggingFace |
| BGE embeddings | BAAI |

Example:

```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")
```

---

## 16. ChromaDB with LangChain

LangChain integrates easily with Chroma.

Example:

```python
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings

vectorstore = Chroma(
    collection_name="docs",
    embedding_function=OpenAIEmbeddings()
)
```

Search:

```python
docs = vectorstore.similarity_search("What is AI?")
```

---

## 17. ChromaDB with RAG

Typical RAG architecture:

```plain text
User Query
   ↓
Embedding Model
   ↓
ChromaDB Search
   ↓
Top Documents
   ↓
LLM Prompt
   ↓
Answer
```

## 18. ChromaDB Persistence

Two modes:

### In-memory

Data disappears when program stops.

```python
client = chromadb.Client()
```

---

### Persistent Storage

Data saved locally.

```python
persist_directory="./chroma"
```

Useful for production systems.

---

## 19. ChromaDB Server Mode

Chroma can run as a server.

Example:

```bash
chroma run
```

Then connect via API.

Benefits:

- multi-user access
- scalable deployments

---

## 20. Performance Optimization

Improve performance by:

- using efficient embeddings
- limiting search results
- metadata filtering
- batching inserts
- indexing large datasets

## 21. ChromaDB vs Other Vector Databases

| Database | Type |
|---|---|
| ChromaDB | lightweight AI vector DB |
| Pinecone | managed vector DB |
| Weaviate | semantic database |
| Milvus | large-scale vector DB |
| FAISS | vector search library |

ChromaDB advantages:

- simple setup
- open source
- developer friendly
- ideal for RAG apps

---

## 22. Applications of ChromaDB

### Retrieval-Augmented Generation (RAG)

document retrieval for LLMs

---

### Semantic Search

search based on meaning rather than keywords

---

### AI Agents

long-term memory storage

---

### Recommendation Systems

content similarity

---

### Knowledge Bases

enterprise search

---

## 23. Security Considerations

Potential risks:

- sensitive data storage
- unauthorized access
- prompt injection via retrieved documents

Mitigation:

- encryption
- access control
- document validation

## 24. Best Practices

1. clean and chunk documents
2. store metadata for filtering
3. use high-quality embeddings
4. limit retrieval results
5. monitor search quality

---

## 25. Example Full Pipeline

Example RAG pipeline with Chroma:

```python
docs = collection.query(
    query_texts=["Explain transformers"],
    n_results=3
)

context = docs["documents"]

prompt = f"""
Use this context to answer:

{context}

Question: Explain transformers
"""
```

---

## 26. ChromaDB Ecosystem

Common integrations:

- LangChain
- LlamaIndex
- OpenAI
- HuggingFace
- FastAPI
- Streamlit

---

## 27. Quick Mental Model

```text
Text → Embedding → Vector → Similarity Search → Retrieved Context
```

or

```text
Knowledge Base
      ↓
Vector Embeddings
      ↓
ChromaDB
      ↓
Relevant Results
```